CISC 886 – Cloud Computing

Project Deliverable

School of Computing, Queen's University, Kingston, Canada

Overview

This project is worth **25% of your final grade**. You will build a complete, end-to-end cloud-based chat assistant, from infrastructure provisioning to a live web interface, deployed individually on AWS. You will submit a written report (PDF) covering all sections below and a GitHub repository containing all code with a README that allows full replication.

> **Resource naming policy:** Every AWS resource you create must be prefixed with your Queen's netID (e.g. `q1abc-vpc`, `q1abc-ec2`). This is required for grading and to avoid conflicts on the shared account.

Section 1 — System Architecture `2 marks`

Provide a diagram showing all components of your deployed system and how they connect. The diagram must include your VPC, subnets, security groups, EMR cluster, EC2 instance, LLM runner, and chat interface. Alongside the diagram, include one paragraph describing how data flows through your system from preprocessing to user interaction.

What to submit: A system architecture diagram (draw.io, Lucidchart, or equivalent) and a written paragraph in the report.

Section 2 — VPC & Networking `4 marks`

You must create a **new VPC** for this project. The default AWS VPC is not allowed. You may provision the VPC using Terraform or the AWS Console; either approach is acceptable as long as you justify your choice. Document your CIDR block and subnet design, your Internet Gateway and route table configuration, and your security group rules, explaining which ports are open, to which sources, and why. For each decision, explain the reasoning behind it, not only what you configured but why you configured it that way.

What to submit: Terraform `.tf` file(s) committed to GitHub, or annotated console screenshots embedded in the report, along with a written justification section.

Section 3 — Model & Dataset Selection `3 marks`

Clearly explain which model and dataset you selected, and why. For the model, document the model name, number of parameters, and a link to its source (Unsloth, Hugging Face, or Ollama). Include the model license and explain why this model is appropriate for your chosen task in terms of domain fit and hardware requirements. For the dataset, document the name, source, and license alongside the number of samples, your train/validation/test split strategy, and how you avoided data leakage. Include at least one sample shown verbatim and summary statistics such as token length distribution or label distribution where applicable.

What to submit: A written report section. No code is required for this section.

Section 4 — Data Preprocessing with Apache Spark on EMR `5 marks`

You must use **AWS EMR** to run a PySpark preprocessing pipeline on your dataset before fine-tuning. Document your EMR cluster configuration including instance type, number of nodes, and region. Your PySpark code must be committed to GitHub with inline explanation of each step, and you must include a screenshot of the preprocessed output files saved to S3. Include at least 3 exploratory data analysis figures (for example, token length distribution, class or label balance, and sample count per split), each with a short caption. You must also provide a screenshot of your EMR cluster in the Terminated state as proof of teardown.

What to submit: PySpark script or notebook in GitHub (not screenshots of code), an EMR console screenshot showing the cluster configuration, an EMR console screenshot showing the cluster in **Terminated** state, an S3 screenshot showing the output files, and EDA figures in the report.

> ⚠️ **The teardown screenshot is required.** A missing teardown screenshot will be treated as an incomplete submission for this section. EMR clusters left running deplete the shared account.

Section 5 — Model Fine-Tuning `6 marks`

Fine-tune your chosen model using a parameter-efficient approach suitable for the hardware available to you, following the workflow described in the project resource guide. Document the library, technique, and hardware you used. Include the full training code with inline explanation committed to GitHub. You must also include a hyperparameter table covering at minimum your learning rate, batch size, number of epochs, and any adapter-specific values. Provide at least 2 example prompts showing the base model response alongside the fine-tuned model response to demonstrate the effect of fine-tuning.

What to submit: A runnable notebook (`.ipynb`) committed to GitHub and a written section in the report documenting your training setup and qualitative comparison.

> **Optional (encouraged):** Hyperparameter table in the report; training loss curve; side-by-side comparison table or screenshots.

Section 6 — Model Deployment on EC2 `3 marks`

Deploy your fine-tuned model to an AWS EC2 instance and serve it using an LLM runner. Document the EC2 instance type and AMI you used, and include the exact commands used to install the runner and load your model. Provide a terminal screenshot showing the runner serving your fine-tuned model with the model name visible, and a `curl` call to the running model API with the response shown.

What to submit: All commands copy-pasted verbatim in both the report and the README, a terminal screenshot of the runner, and a screenshot of the `curl` response from the model.

Section 7 — Web Interface `2 marks`

Connect your deployed model to a web-based chat interface so it can be accessed from a browser. The interface must be configured to start automatically when the server restarts. Provide a screenshot of the interface running in a browser with your fine-tuned model name visible, and a screenshot of a sample conversation with the model through the interface.

What to submit: Screenshots in the report showing the live interface and a sample conversation.

GitHub Repository Requirements *(required for passing)*

Your repository must include all code for every section, including your PySpark script, fine-tuning notebook, and Terraform files if used. The repository must also contain a `README.md` with prerequisites (tools, accounts, AWS region), step-by-step commands to replicate each phase of the project end to end, and a cost summary table showing your approximate AWS spend per service.

> A submission without a working README that covers replication steps is considered incomplete for all sections that involve code.

Mark Summary

#	Section	Marks
1	System Architecture	2
2	VPC & Networking	4
3	Model & Dataset Selection	3
4	Data Preprocessing (EMR + Spark)	5
5	Model Fine-Tuning	6
6	Model Deployment on EC2	3
7	Web Interface	2
	Total	**25**